Basic + WHERE + Operators (1–15)

```
select * from accounts where balance>20000;
select * from customers where city = 'Chennai';
select * from accounts where balance between 20000 and 50000;
select * from customers where name like 'J%';
select * from accounts where account_type in ('Savings', 'Current');
select * from customers where name like '%a%';
select * from accounts where balance <= 30000;
select * from customers where city not in 'Madurai';
select * from accounts where balance not between 10000 and 40000;
select * from customers where name like '%i';
select * from accounts where balance = 50000;
select * from customers where city in ('Chennai', 'Salem');
select * from accounts where balance > 10000 and balance < 40000;
select * from accounts where account_type not in ('Current');
```

ORDER BY (16–18)

```
select * from accounts order by balance desc;
select * from customers order by name asc;
select * from accounts order by account_type desc , balance asc;
```

Aggregate Functions (19–23)

```
select sum(balance) as total_balance from accounts;
select avg(balance) as average_balance from accounts;
select max(balance) as maximum_balance from accounts;
select min(balance) as minimum_balance from accounts;
select count(*) as total_customers from customers;
```

GROUP BY + HAVING (24–28)

```
select account_type, sum(balance) as total_balance from accounts group by account_type;
select account_type, avg(balance) as average_balance from accounts group by account_type;
select account_type, avg(balance) as average_balance from accounts group by account_type
having avg(balance) > 20000;
select customer_id, count(*) as total_accounts from accounts group by customer_id;
```

```
select customer_id, count(*) as total_accounts from accounts group by customer_id having count(*)
> 1;
JOINS (29–35)
select c.name, a.balance from customers c inner join accounts a on c.customer_id = a.customer_id;
select c.*, a.* from customers c left join accounts a on c.customer_id = a.customer_id;
select a.*, c.* from accounts a inner join customers c on a.customer_id = c.customer_id;
select c.name, a.account_type, a.balance from customers c inner join accounts a on c.customer_id =
a.customer_id where a.balance > 20000;
select c.customer_id, c.name, sum(a.balance) as total_balance from customers c left join accounts a
on c.customer_id = a.customer_id group by c.customer_id, c.name;
select c.name, a.balance from customers c inner join accounts a on c.customer_id = a.customer_id
order by a.balance asc;
select c.city, count(a.account_id) as total_accounts from customers c left join accounts a
on c.customer_id = a.customer_id group by c.city;
-- SUBQUERIES (36–40)
select * from accounts where balance > (select avg(balance) from accounts);
select * from customers where customer_id in (select customer_id from accounts);
select * from customers where customer_id not in (select customer_id from accounts where
customer_id is not null);
select * from accounts where balance = (select max(balance) from accounts);
select * from customers where customer_id in (select customer_id from accounts group by
customer_id having sum(balance) > 40000);
-- RAILWAY RESERVATION
--WHERE + Operators (41–45)
select * from bookings where fare > 400;
select * from bookings where status <> 'Confirmed';
select * from trains where source = 'Chennai';
select * from bookings where fare between 300 and 500;
select * from bookings where passenger_name like 'A%';
-- JOINS + GROUP BY + ORDER BY (46–48)
select t.train_name, b.passenger_name from trains t inner join bookings b on t.train_id = b.train_id;
```

```
select t.train_name, count(b.booking_id) as total_bookings from trains t left join bookings b on t.train_id = b.train_id group by t.train_id, t.train_name;
select t.train_name, sum(b.fare) as total_fare from trains t inner join bookings b on t.train_id = b.train_id group by t.train_id, t.train_name;
-- SUBQUERIES (49–50)
select * from bookings where fare = (select max(fare) from bookings);
select * from trains where train_id in (select train_id from bookings group by train_id having count(*) > 1);
-- Employees Table (1–15)
select department, count(*) as total_employees from employees group by department;
select department, avg(salary) as average_salary from employees group by department;
select department, count(*) as total_employees from employees group by department having count(*) > 1;
select department, max(salary) as highest_salary from employees group by department;
select department, min(salary) as lowest_salary from employees group by department;
select department, avg(salary) as average_salary from employees group by department having avg(salary) > 50000;
select department, sum(salary) as total_salary from employees group by department;
select * from employees order by salary desc;
select * from employees order by department asc, salary desc;
select city, count(*) as total_employees from employees group by city having count(*) > 1;
select city, sum(salary) as total_salary from employees group by city;
select department, sum(salary) as total_salary from employees group by department order by total_salary desc;
select department, count(*) as total_employees from employees where salary > 50000 group by department;
select department, max(salary) - min(salary) as salary_difference from employees group by department;
select * from employees order by salary desc limit 3;
-- Customers & Orders Tables (16–30)
select customer_id, sum(amount) as total_amount from orders group by customer_id;
```

```
select customer_id, count(*) as total_orders from orders group by customer_id having count(*) > 3;
select customer_id, avg(amount) as average_amount from orders group by customer_id;
select customer_id, max(amount) as highest_order from orders group by customer_id;
select customer_id, sum(amount) as total_purchase from orders group by customer_id order by
total_purchase desc;
select customer_id, sum(amount) as total_purchase from orders group by customer_id having
sum(amount) > 10000;
select c.name, count(o.order_id) as total_orders from customers c left join orders o on c.customer_id
= o.customer_id group by c.customer_id, c.name;
select customer_id, sum(amount) as total_purchase from orders group by customer_id order by
total_purchase desc limit 1;
select customer_id, count(*) as total_orders from orders group by customer_id order by total_orders
desc limit 1;
select customer_id, avg(amount) as average_amount from orders group by customer_id having
avg(amount) > 2000;
select customer_id, sum(amount) as total_purchase from orders group by customer_id order by
total_purchase desc limit 5;
select customer_id, min(amount) as minimum_order from orders group by customer_id;
select customer_id, sum(amount) as total_purchase from orders group by customer_id having
sum(amount) > 5000;
select customer_id, count(*) as total_orders, sum(amount) as total_purchase from orders group by
customer_id;
select customer_id, count(*) as total_orders, sum(amount) as total_purchase from orders group by
customer_id having count(*) > 2 and sum(amount) > 8000;
-- Students Table (31–45)
select department, avg(marks) as average_marks from students group by department;
select department, avg(marks) as average_marks from students group by department having
avg(marks) > 75;
select department, max(marks) as highest_marks from students group by department;
select department, count(*) as total_students from students group by department;
select department, count(*) as total_students from students group by department having count(*) >
```

```
5;
select department, avg(marks) as average_marks from students group by department order by
average_marks desc;
select department, avg(marks) as average_marks from students group by department order by
average_marks desc limit 3;
select department, avg(marks) as average_marks from students group by department having
avg(marks) between 70 and 90;
select department, sum(marks) as total_marks from students group by department;
select department, count(*) as total_students from students group by department order by
total_students desc;
select department, min(marks) as lowest_marks from students group by department;
select department, max(marks) as highest_marks from students group by department having
max(marks) > 90;
select department, count(*) as total_students from students where marks > 80 group by
department;
select department, count(*) as total_students from students where marks > 75 group by department
having count(*) > 3;
select department, max(marks) as highest_marks from students group by department order by
highest_marks desc;
```